UIDAI Aadhaar Service

TEAM MEMBERS :→

DAKSH SINGH (TEAM LEADER)

ADITYA KUMAR

ANSHUMAN BISWAS

DIVYANSH NAGAR

SAKSHAM SINGH

Objective

This project aims to transform raw Aadhaar enrolment and update data into **actionable, district-level intelligence** for policy planning.

By integrating enrolment, demographic updates, and biometric updates at a district**–month granularity**, we identify **future service pressure, lifecycle stress, demographic balance, and system utilization efficiency** across India.

Data & Methodology

- **Data Sources:**
  - Aadhaar Enrolment Data
  - Demographic Update Data
  - Biometric Update Data
- **Processing Scale:**
  - Approximately **5 million raw records** cleaned and standardized
  - Aggregated to **4,730 unique district–month observations**
- **Key Steps:**
  - Data cleaning & normalization (state/district standardization)
  - District–month aggregation to avoid double counting
  - Defensive re-aggregation to ensure join integrity
  - Unified master analytical dataset creation

Key Analytical Features (Core Contribution)

1. Transition Pressure Index (TPI)

Measures **future Aadhaar service pressure** by comparing the population aged **5–17** with current adult update activity.

→ Identifies districts likely to face **future biometric and demographic update surges**.

---

2. Child Lifecycle Stress Indicator (CLSI)

Quantifies how much Aadhaar update demand is driven by **children (5–17)**.

→ Highlights districts where **child-centric lifecycle updates dominate**, requiring specialized service planning.

---

3. Demographic Balance Score (DBS)

Evaluates whether Aadhaar enrolment is **balanced across age groups**.

→ Detects districts with **skewed adoption** (only adults or only children), enabling targeted enrollment drives.

---

4. Conversion Efficiency Score (CES)

Measures how effectively enrolment translates into **real system engagement** (updates per enrolled citizen).

→ Identifies **active vs dormant Aadhaar regions**, separating enrollment quantity from operational usage.

Key Insights

- Most districts show **low to moderate future pressure**, while a **small subset faces extreme upcoming demand**
- Several regions have **high enrolment but low engagement**, indicating operational dormancy
- Child-driven update regions often show **higher system maturity**
- Demographic balance does **not automatically imply high system usage**
- Lifecycle-based metrics reveal patterns **not visible through raw counts alone**

Policy & Operational Value

This framework enables UIDAI and administrators to:

- Anticipate **future service bottlenecks**
- Identify **child-heavy lifecycle demand districts**
- Detect **imbalanced or exclusionary enrollment patterns**
- Distinguish **active vs underutilized Aadhaar regions**
- Design **targeted, age-specific and region-specific interventions**

MODIFIED DATASET LINK :→ https://drive.google.com/file/d/1kv7cZIjWWIufuGahI-RoFkMekWaM3345/view?usp=sharing

NOTEBOOK LINK :→ https://drive.google.com/file/d/1z2F9Ge1_P9QRP-OzTr3TPrK5PY9IJr6x/view?usp=sharing

DATA CLEANING

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import glob
pd.set_option("display.max_columns", None)
```

```
bio_files = glob.glob("data/biometric/*.csv")
demo_files = glob.glob("data/demographic/*.csv")
enr_files = glob.glob("data/enrolment/*.csv")

print("Biometric files:", len(bio_files))
print("Demographic files:", len(demo_files))
print("Enrolment files:", len(enr_files))
```

Biometric files: 4
Demographic files: 5
Enrolment files: 3

```
bio_dfs = []
for file in bio_files:
    df = pd.read_csv(file)
    bio_dfs.append(df)

biometric_base = pd.concat(bio_dfs, ignore_index=True)
print("Biometric data shape:", biometric_base.shape)
biometric_base.head()
```

Biometric data shape: (1861108, 6)

	date	state	district	pincode	bio_age_5_17	bio_age_17_
0	01-03-2025	Haryana	Mahendragarh	123029	280	577
1	01-03-2025	Bihar	Madhepura	852121	144	369
2	01-03-2025	Jammu and Kashmir	Punch	185101	643	1091
3	01-03-2025	Bihar	Bhojpur	802158	256	980
4	01-03-2025	Tamil Nadu	Madurai	625514	271	815

```
demo_dfs = []

for file in demo_files:
    df = pd.read_csv(file)
    demo_dfs.append(df)

demographic_base = pd.concat(demo_dfs, ignore_index=True)
```

```
print("Demographic data shape:", demographic_base.shape)
demographic_base.head()
```

Demographic data shape: (2071700, 6)

	date	state	district	pincode	demo_age_5_17	demo_age_17_
0	01-03-2025	Uttar Pradesh	Gorakhpur	273213	49	529
1	01-03-2025	Andhra Pradesh	Chittoor	517132	22	375
2	01-03-2025	Gujarat	Rajkot	360006	65	765
3	01-03-2025	Andhra Pradesh	Srikakulam	532484	24	314
4	01-03-2025	Rajasthan	Udaipur	313801	45	785

```
enr_dfs = []

for file in enr_files:
    df = pd.read_csv(file)
    enr_dfs.append(df)

enrolment_base = pd.concat(enr_dfs, ignore_index=True)

print("Enrolment data shape:", enrolment_base.shape)
enrolment_base.head()
```

Enrolment data shape: (1006029, 7)

	date	state	district	pincode	age_0_5	age_5_17	age_18_great
0	02-03-2025	Meghalaya	East Khasi Hills	793121	11	61	37
1	09-03-2025	Karnataka	Bengaluru Urban	560043	14	33	39
2	09-03-2025	Uttar Pradesh	Kanpur Nagar	208001	29	82	12
3	09-03-2025	Uttar Pradesh	Aligarh	202133	62	29	15
4	09-03-2025	Karnataka	Bengaluru Urban	560016	14	16	21

```
for df in [biometric_base, demographic_base, enrolment_base]:
    df.columns = df.columns.str.lower().str.strip()
```

```
for df in [biometric_base, demographic_base, enrolment_base]:
    df['date'] = pd.to_datetime(
        df['date'],
        format='mixed',
        dayfirst=True,
        errors='coerce'
    )
    df['year'] = df['date'].dt.year
    df['month'] = df['date'].dt.month
```

```
for df in [biometric_base, demographic_base, enrolment_base]:
    df['state'] = df['state'].str.lower().str.strip()
    df['district'] = df['district'].str.lower().str.strip()
```

```
demographic_base.head()
```

	date	state	district	pincode	demo_age_5_17	demo_age_17_	year
0	2025-03-01	uttar pradesh	gorakhpur	273213	49	529	2025
1	2025-03-01	andhra pradesh	chittoor	517132	22	375	2025
2	2025-03-01	gujarat	rajkot	360006	65	765	2025
3	2025-03-01	andhra pradesh	srikakulam	532484	24	314	2025
4	2025-03-01	rajasthan	udaipur	313801	45	785	2025

District–Month Aggregation

To align the analysis with administrative decision-making levels and avoid double counting, all datasets are independently aggregated to a district–month level.

```
enrolment_dm = (
    enrolment_base
    .groupby(['state', 'district', 'year', 'month'], as_index=False)
    .agg({
        'age_0_5': 'sum',
        'age_5_17': 'sum',
        'age_18_greater': 'sum'
    })
)

print("Enrolment aggregated shape:", enrolment_dm.shape)
enrolment_dm.head()
```

Enrolment aggregated shape: (5004, 7)

	state	district	year	month	age_0_5	age_5_17	age_18_great
0	100000	100000	2025	9	0	0	12
1	100000	100000	2025	10	0	1	0
2	100000	100000	2025	11	0	0	11
3	100000	100000	2025	12	0	0	194
4	andaman & nicobar islands	andamans	2025	9	23	4	0

```
demographic_dm = (
    demographic_base
    .groupby(['state', 'district', 'year', 'month'], as_index=False)
    .agg({
        'demo_age_5_17': 'sum',
        'demo_age_17_': 'sum'
    })
)
```

```
print("Demographic aggregated shape:", demographic_dm.shape)
demographic_dm.head()
```

Demographic aggregated shape: (5954, 6)

	state	district	year	month	demo_age_5_17	demo_age_17_
0	100000	100000	2025	12	0	2
1	andaman & nicobar islands	andamans	2025	9	3	159
2	andaman & nicobar islands	andamans	2025	10	2	73
3	andaman & nicobar islands	andamans	2025	11	0	212
4	andaman & nicobar islands	andamans	2025	12	2	299

```
biometric_dm = (
    biometric_base
    .groupby(['state', 'district', 'year', 'month'], as_index=False)
    .agg({
        'bio_age_5_17': 'sum',
        'bio_age_17_': 'sum'
    })
)

print("Biometric aggregated shape:", biometric_dm.shape)
biometric_dm.head()
```

Biometric aggregated shape: (8383, 6)

	state	district	year	month	bio_age_5_17	bio_age_17_
0	andaman & nicobar islands	andamans	2025	3	16	193
1	andaman & nicobar islands	andamans	2025	4	17	167
2	andaman & nicobar islands	andamans	2025	5	22	158
3	andaman & nicobar islands	andamans	2025	6	11	129
4	andaman & nicobar islands	andamans	2025	7	20	239

Remove numeric-coded state/district rows

```
enrolment_dm = enrolment_dm[
    (~enrolment_dm['state'].str.isnumeric()) &
    (~enrolment_dm['district'].str.isnumeric())
].copy()
```

```
enrolment_dm['state'].str.isnumeric().sum(), enrolment_dm['district'].str.isnumeric().sum()
```

(np.int64(0), np.int64(0))

```
demographic_dm = demographic_dm[
    (~demographic_dm['state'].str.isnumeric()) &
    (~demographic_dm['district'].str.isnumeric())
].copy()
```

```
demographic_dm['state'].str.isnumeric().sum(), demographic_dm['district'].str.isnumeric().sum()
```

(np.int64(0), np.int64(0))

```
biometric_dm = biometric_dm[
    (~biometric_dm['state'].str.isnumeric()) &
    (~biometric_dm['district'].str.isnumeric())
].copy()
```

```
biometric_dm['state'].str.isnumeric().sum(), biometric_dm['district'].str.isnumeric().sum()
```

(np.int64(0), np.int64(0))

```
print(enrolment_dm['state'].str.isnumeric().sum(),
      demographic_dm['state'].str.isnumeric().sum(),
      biometric_dm['state'].str.isnumeric().sum())
```

0 0 0

```
print(
    enrolment_dm.duplicated(['state','district','year','month']).sum(),
    demographic_dm.duplicated(['state','district','year','month']).sum(),
    biometric_dm.duplicated(['state','district','year','month']).sum()
)
```

0 0 0

```
print("Raw enrolment totals:")
print(enrolment_base[['age_0_5','age_5_17','age_18_greater']].sum())

print("\nAggregated enrolment totals:")
print(enrolment_dm[['age_0_5','age_5_17','age_18_greater']].sum())
```

Raw Enrolment Totals

Age Group	Enrolled Count
age_0_5	3,546,965
age_5_17	1,720,384
age_18_greater	168,353

Aggregated enrolment totals:

Age Group	Enrolled Count
age_0_5	3,546,965
age_5_17	1,720,383
age_18_greater	168,136

```
for df in [enrolment_dm, demographic_dm, biometric_dm]:
    if 'date' in df.columns:
        df.drop(columns=['date'], inplace=True)
```

```
import re
def normalize_text(x):
    x = x.lower().strip()
    x = x.replace('&', 'and')
    x = re.sub(r'[^a-z\s]', '', x)
    x = " ".join(x.split())
    return x
```

```
for df in [enrolment_dm, demographic_dm, biometric_dm]:
    df['state'] = df['state'].apply(normalize_text)
    df['district'] = df['district'].apply(normalize_text)
```

```
for df in [enrolment_dm, demographic_dm, biometric_dm]:
    df['state'] = df['state'].apply(standardize_state)
```

```
for df in [enrolment_dm, demographic_dm, biometric_dm]:
    print(sorted(df['state'].unique()))
```

```
for df in [enrolment_dm, demographic_dm, biometric_dm]:
    df.drop(df[~df['state'].isin(VALID_STATES)].index, inplace=True)
```

```
for df in [enrolment_dm, demographic_dm, biometric_dm]:
    print(sorted(df['state'].unique()))
```

To correct inconsistent district and state names in the anonymised Aadhaar dataset, a **reference-based fuzzy string matching approach** was adopted. Instead of relying on exact string equality, similarity scores were computed between observed location names and official reference names to identify the most likely correct match.

To ensure scalability and accuracy, fuzzy matching was applied only to **unique location values** and constrained within the corresponding state-level reference lists. Automatic correction was performed only for **high-confidence matches**, while low-confidence or ambiguous cases were flagged and excluded from auto-correction to prevent erroneous mappings.

This approach enables efficient standardization of large datasets while maintaining transparency, auditability, and alignment with UIDAI's data governance principles.

Data Integration

The independently aggregated enrolment, demographic update, and biometric update datasets are integrated using district–month keys to create a unified analytical table.

During integration, duplicate district–month records were detected due to residual multi-row aggregates in source datasets. To ensure join integrity, datasets were defensively re-aggregated before final integration.

```
def force_unique(df, value_cols):
    return (
        df.groupby(['state', 'district', 'year', 'month'], as_index=False)
          .agg({col: 'sum' for col in value_cols})
    )
```

```
enrolment_dm = force_unique(
    enrolment_dm,
    ['age_0_5', 'age_5_17', 'age_18_greater']
)
```

```
demographic_dm = force_unique(
    demographic_dm,
    ['demo_age_5_17', 'demo_age_17_']
)
```

```
biometric_dm = force_unique(
    biometric_dm,
    ['bio_age_5_17', 'bio_age_17_']
)
```

```
# Check keys exist and types look right
for name, df in zip(
    ['enrolment_dm', 'demographic_dm', 'biometric_dm'],
    [enrolment_dm, demographic_dm, biometric_dm]
):
    print(name, df[['state','district','year','month']].isna().sum().to_dict())
```

```
enrolment_dm {'state': 0, 'district': 0, 'year': 0, 'month': 0}
demographic_dm {'state': 0, 'district': 0, 'year': 0, 'month': 0}
biometric_dm {'state': 0, 'district': 0, 'year': 0, 'month': 0}
```

```
master_df = enrolment_dm.merge(
    demographic_dm,
    on=['state', 'district', 'year', 'month'],
    how='left'
)

print("After joining demographic:", master_df.shape)
master_df.head()
```

```
After joining demographic: (4730, 9)
```

	state	district	year	month	age_0_5	age_5_17	age_18_great
0	andaman and nicobar islands	andamans	2025	9	23	4	0
1	andaman and nicobar islands	andamans	2025	10	15	0	0
2	andaman and nicobar islands	andamans	2025	11	13	0	0
3	andaman and nicobar islands	andamans	2025	12	19	1	0
4	andaman and nicobar islands	nicobar	2025	9	41	6	0

```
master_df = master_df.merge(
    biometric_dm,
    on=['state', 'district', 'year', 'month'],
    how='left'
)

print("After joining biometric:", master_df.shape)
master_df.head()
```

After joining biometric: (4730, 11)

	state	district	year	month	age_0_5	age_5_17	age_18_great
0	andaman and nicobar islands	andamans	2025	9	23	4	0
1	andaman and nicobar islands	andamans	2025	10	15	0	0
2	andaman and nicobar islands	andamans	2025	11	13	0	0
3	andaman and nicobar islands	andamans	2025	12	19	1	0
4	andaman and nicobar islands	nicobar	2025	9	41	6	0

```
# Missing updates mean zero activity
master_df.fillna(0, inplace=True)
```

```
dup_count = master_df.duplicated(['state','district','year','month']).sum()
dup_count
```

np.int64(0)

```
master_df.columns
```

```
Index(['state', 'district', 'year', 'month', 'age_0_5', 'age_5_17',
'age_18_greater', 'demo_age_5_17', 'demo_age_17_', 'bio_age_5_17',
'bio_age_17_'],
dtype='object')
```

```
master_df.describe()
```

	year	month	age_0_5	age_5_17	age_18_greater	demo_age_5_17	demo_age_17
count	4730.0	4730.000000	4730.000000	4730.000000	4730.000000	4730.000000	4730.00000
mean	2025.0	9.340803	749.886892	363.717336	35.546723	746.631078	6859.818816
std	0.0	2.428971	871.748807	595.834389	107.067932	1000.817176	10345.09091
min	2025.0	3.000000	0.000000	0.000000	0.000000	0.000000	0.000000
25%	2025.0	9.000000	145.000000	35.000000	1.000000	43.000000	467.750000
50%	2025.0	10.000000	488.000000	134.000000	9.000000	393.500000	3368.00000
75%	2025.0	11.000000	1039.000000	413.750000	28.000000	1015.750000	8503.00000
max	2025.0	12.000000	12270.000000	6314.000000	2404.000000	9362.000000	131818.0000

```
count_cols = [
    'demo_age_5_17','demo_age_17_',
    'bio_age_5_17','bio_age_17_'
]

master_df[count_cols] = master_df[count_cols].astype(int)
```

```
master_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4730 entries, 0 to 4729
Data columns (total 11 columns):
```

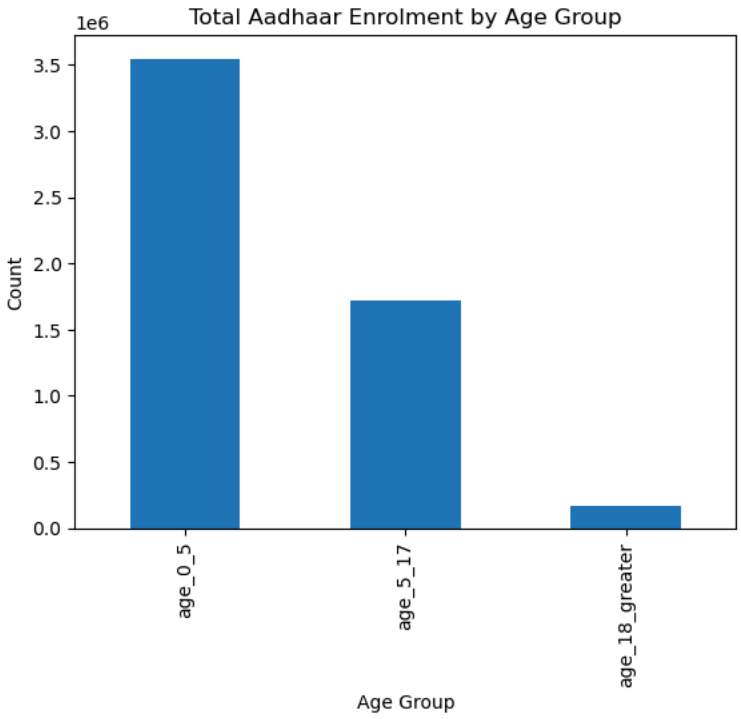
#	Column	Non-Null Count	Dtype
0	state	4730 non-null	object
1	district	4730 non-null	object
2	year	4730 non-null	int32
3	month	4730 non-null	int32
4	age_0_5	4730 non-null	int64
5	age_5_17	4730 non-null	int64
6	age_18_greater	4730 non-null	int64
7	demo_age_5_17	4730 non-null	int64
8	demo_age_17_	4730 non-null	int64
9	bio_age_5_17	4730 non-null	int64
10	bio_age_17_	4730 non-null	int64

```
dtypes: int32(2), int64(7), object(2)
memory usage: 369.7+ KB
```

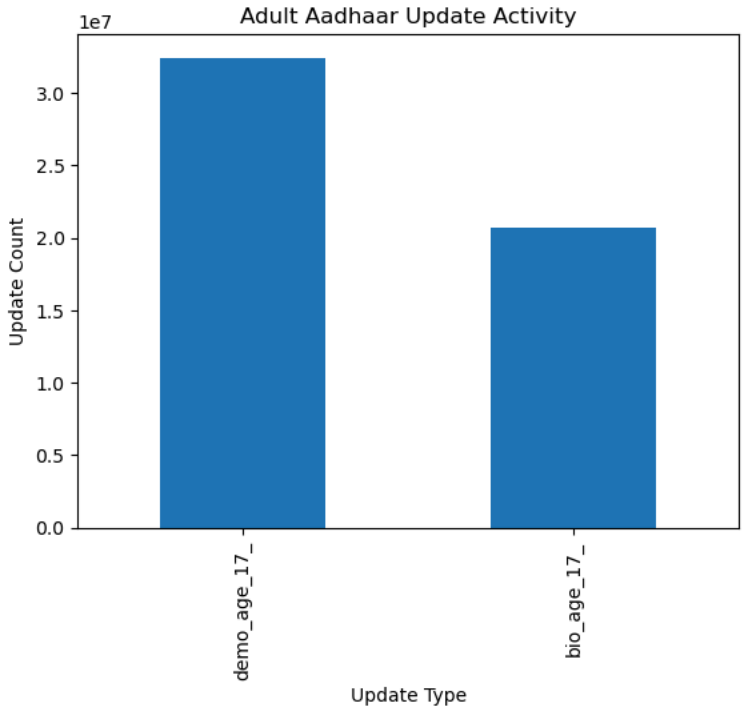
```
import os

os.makedirs("processed", exist_ok=True)
master_df.to_csv(
    "processed/master_district_month.csv",
    index=False
)
```

Exploratory Data Analysis

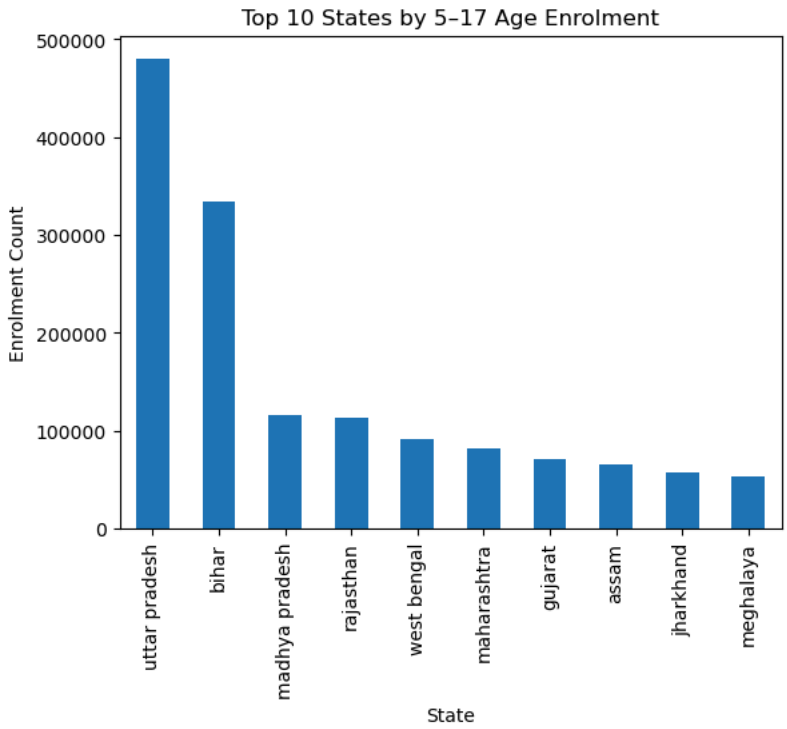
```
#PLOT 1 — Enrolment age distribution
age_cols = ['age_0_5', 'age_5_17', 'age_18_greater']

master_df[age_cols].sum().plot(kind='bar')
plt.title('Total Aadhaar Enrolment by Age Group')
plt.ylabel('Count')
plt.xlabel('Age Group')
plt.show()
```



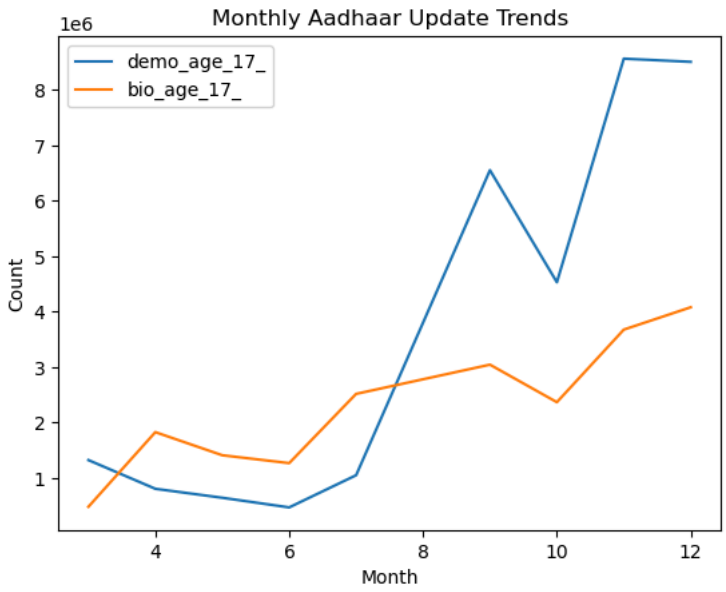
```
#PLOT 2 — Demographic vs Biometric updates(SYSTEM LOAD)
update_cols = ['demo_age_17_', 'bio_age_17_']

master_df[update_cols].sum().plot(kind='bar')
plt.title('Adult Aadhaar Update Activity')
plt.ylabel('Update Count')
plt.xlabel('Update Type')
plt.show()
```



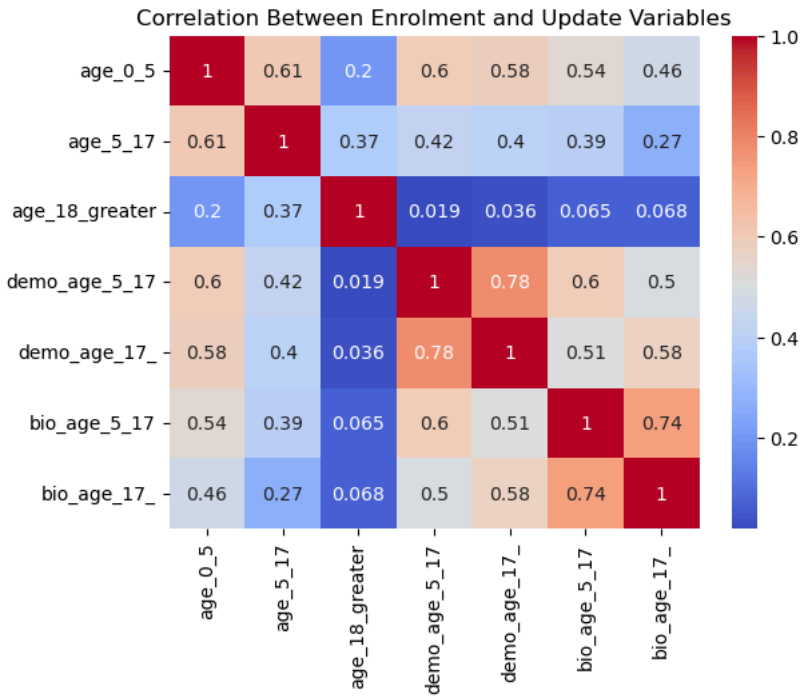

```
#PLOT 3 — State-wise enrolment load (INEQUALITY SIGNAL)
state_enrolment = (
    master_df
    .groupby('state')['age_5_17']
    .sum()
    .sort_values(ascending=False)
    .head(10)
)

state_enrolment.plot(kind='bar')
plt.title('Top 10 States by 5–17 Age Enrolment')
plt.ylabel('Enrolment Count')
plt.xlabel('State')
plt.show()
```




```
#PLOT 4 — Monthly trend of updates (TEMPORAL DYNAMICS)
monthly_updates = (
    master_df
    .groupby('month')[['demo_age_17_', 'bio_age_17_']]
    .sum()
)

monthly_updates.plot()
plt.title('Monthly Aadhaar Update Trends')
plt.ylabel('Count')
plt.xlabel('Month')
plt.show()
```




```
#PLOT 5 — Correlation heatmap
import seaborn as sns

corr_cols = [
    'age_0_5','age_5_17','age_18_greater',
    'demo_age_5_17','demo_age_17_',
    'bio_age_5_17','bio_age_17_'
]

sns.heatmap(master_df[corr_cols].corr(), annot=True, cmap='coolwarm')
plt.title('Correlation Between Enrolment and Update Variables')
plt.show()
```



Feature Engineering

Feature 1: Transition Pressure Index (TPI)

As Aadhaar enrolment progresses through the lifecycle, individuals enrolled as children (ages 5–17) are expected to transition into the adult biometric update cycle upon turning 18. The Transition Pressure Index (TPI) is designed to estimate future service pressure at the district level by comparing the size of the transitioning population with current adult update activity.

District identifiers are used as reported in the UIDAI datasets to preserve administrative granularity, acknowledging district reorganizations and naming variations over time.

Feature Logic

A district is expected to face higher future Aadhaar service pressure if:

- It has a large population in the 5–17 age group, and
- Current adult update activity is relatively low

TPI captures this imbalance as a forward-looking indicator.

Mathematical Definition

$$\text{TPI} = \frac{\text{Population_aged_5_17}}{\text{Adult_update_activity} + 1}$$

Where adult update activity includes demographic and biometric updates.
The constant +1 avoids division by zero.

```
master_df['TPI'] = (
    master_df['age_5_17'] /
    (master_df['demo_age_17_'] + master_df['bio_age_17_'] + 1)
)
```

Statistic	Value
Count	4730.000000
Mean	11.314123
Std	188.930186
Min	0.000000
25%	0.009139
50%	0.022987
75%	0.053271
Max	6314.000000

```
master_df[['state','district','year','month','age_5_17',
           'demo_age_17_','bio_age_17_','TPI']].head(10)
```

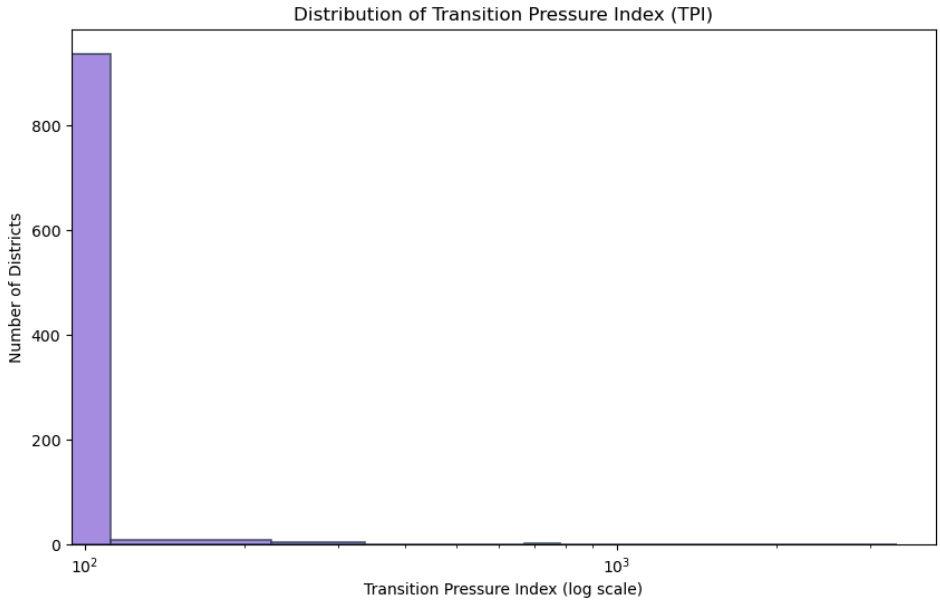
	state	district	year	month	age_5_17	demo_age_17_	bio_age_17_
0	andaman and nicobar islands	andamans	2025	9	4	159	241
1	andaman and nicobar islands	andamans	2025	10	0	73	139
2	andaman and nicobar islands	andamans	2025	11	0	212	174
3	andaman and nicobar islands	andamans	2025	12	1	299	232
4	andaman and nicobar islands	nicobar	2025	9	6	149	155
5	andaman and nicobar islands	nicobar	2025	10	4	50	63
6	andaman and nicobar islands	nicobar	2025	11	0	105	58
7	andaman and nicobar islands	nicobar	2025	12	1	65	53
8	andaman and nicobar islands	nicobars	2025	9	0	0	0
9	andaman and nicobar islands	north and middle andaman	2025	9	2	240	187

```
tpi_district = (
    master_df
    .groupby(['state', 'district'])['TPI']
    .mean()
    .reset_index()
    .sort_values('TPI', ascending=False)
)
```

```
tpi_district.head(10)
```

	state	district	TPI
146	bihar	purbi champaran	3357.000000
369	karnataka	bengaluru urban	2389.000000
922	west bengal	dinajpur uttar	1619.666667
143	bihar	pashchim champaran	1528.729131
888	uttar pradesh	siddharth nagar	1293.000000
945	west bengal	paraganas north	1229.000000
886	uttar pradesh	shravasti	785.000000
859	uttar pradesh	kushi nagar	777.000000
917	west bengal	coochbehar	695.666667
431	madhya pradesh	ashoknagar	661.500000

```
#“Most districts exhibit low to moderate transition pressure, while a small subset shows disproportionately high future d
emand.
plt.figure(figsize=(10, 6))
plt.hist(tpi_district['TPI'], bins=30, color='mediumpurple', edgecolor='#2C3E50', alpha=0.8, linewidth=1.2)
plt.xscale('log')
plt.xlabel('Transition Pressure Index (log scale)')
plt.ylabel('Number of Districts')
plt.title('Distribution of Transition Pressure Index (TPI)')
plt.show()
```

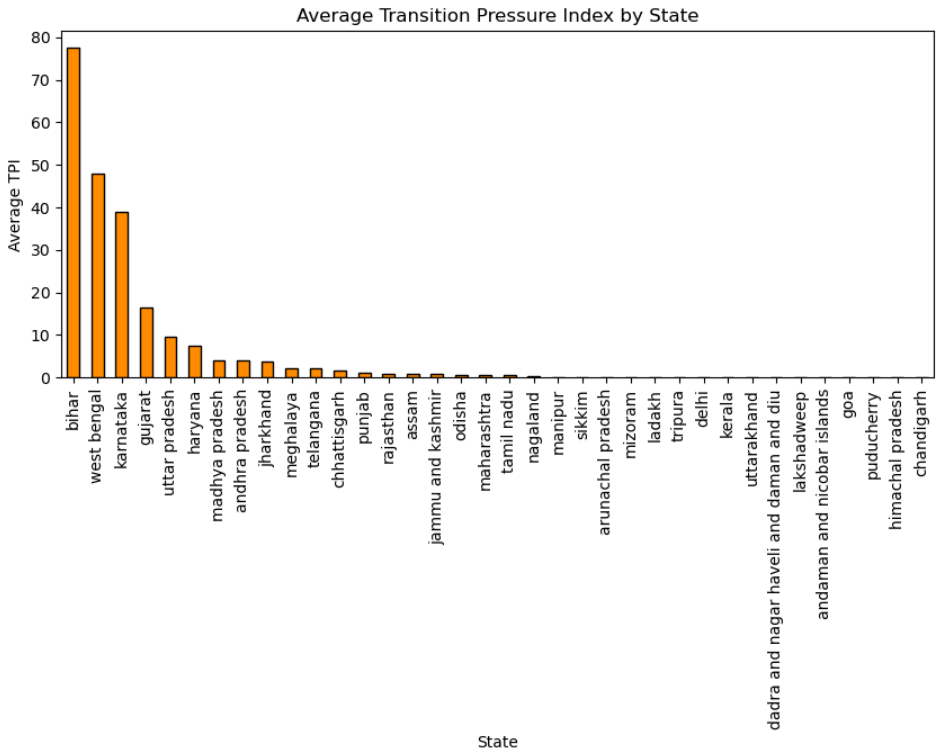



```
tpi_state = (
    master_df
    .groupby('state')['TPI']
    .mean()
    .reset_index()
    .sort_values('TPI', ascending=False)
)

tpi_state.head(10)
```

	state	TPI
4	bihar	77.581143
35	west bengal	47.983449
15	karnataka	38.839497
10	gujarat	16.344974
33	uttar pradesh	9.551406
11	haryana	7.489039
19	madhya pradesh	4.061263
1	andhra pradesh	4.036532
14	jharkhand	3.663148
22	meghalaya	2.299300

```
tpi_state.plot(
    x='state',
    y='TPI',
    kind='bar',
    legend=False,
```

```
    figsize=(10,4),
    color='darkorange',
    edgecolor='black'
)
plt.title('Average Transition Pressure Index by State')
plt.ylabel('Average TPI')
plt.xlabel('State')
plt.show()
```



Feature 2: Child Lifecycle Stress Indicator (CLSI)

Children enrolled in Aadhaar require multiple updates across their lifecycle, including biometric and demographic changes. The Child Lifecycle Stress Indicator (CLSI) quantifies the proportion of Aadhaar update activity attributable to the child population, highlighting districts where lifecycle-driven updates dominate service demand.

Feature Logic

A district experiences higher lifecycle stress if:

- A large share of its total Aadhaar updates are related to children (ages 5–17)

CLSI captures this proportion independent of absolute volume, enabling fair comparison across districts and states.

The Child Lifecycle Stress Indicator reveals districts and states where Aadhaar service demand is disproportionately driven by child-related updates, suggesting the need for age-specialized service planning and targeted awareness interventions.

Mathematical Definition

$$\text{CLSI} = \frac{\text{Child_related_updates}}{\text{Total_updates}}$$

Where: Child-related updates = biometric + demographic updates for ages 5–17

Total updates = all biometric and demographic updates

```
master_df['Child_Related_Updates'] = (
    master_df['bio_age_5_17'] +
    master_df['demo_age_5_17']
)

master_df['Total_Updates_All'] = (
    master_df['bio_age_5_17'] +
    master_df['bio_age_17_'] +
    master_df['demo_age_5_17'] +
    master_df['demo_age_17_']
)

master_df['CLSI'] = master_df['Child_Related_Updates'] / (master_df['Total_Updates_All'] + 1)
```

```
master_df['CLSI'].describe()
```

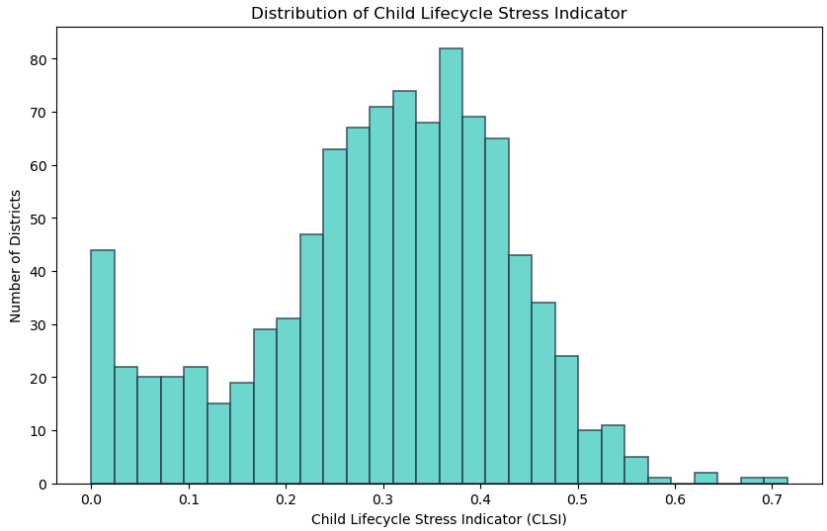
Statistic	Value
Count	4730.000000
Mean	0.315689
Std	0.157211
Min	0.000000
25%	0.213707
50%	0.304308
75%	0.406632
Max	0.871868

```
clsi_district = (
    master_df
    .groupby(['state', 'district'])['CLSI']
    .mean()
    .reset_index()
```

```
    .sort_values('CLSI', ascending=False)
)

clsi_district.head(10)
# These districts are child-update–dominated, even if total load is low.
```

	state	district	CLSI
305	jammu and kashmir	budgam	0.715548
320	jammu and kashmir	ramban	0.681383
59	arunachal pradesh	kurung kumey	0.637212
71	arunachal pradesh	tawang	0.629371
890	uttar pradesh	sitapur	0.575473
324	jammu and kashmir	shupiyan	0.567806
235	gujarat	devbhumi dwarka	0.554953
64	arunachal pradesh	lower siang	0.553724
74	arunachal pradesh	upper subansiri	0.553204
757	tamil nadu	villupuram	0.552109

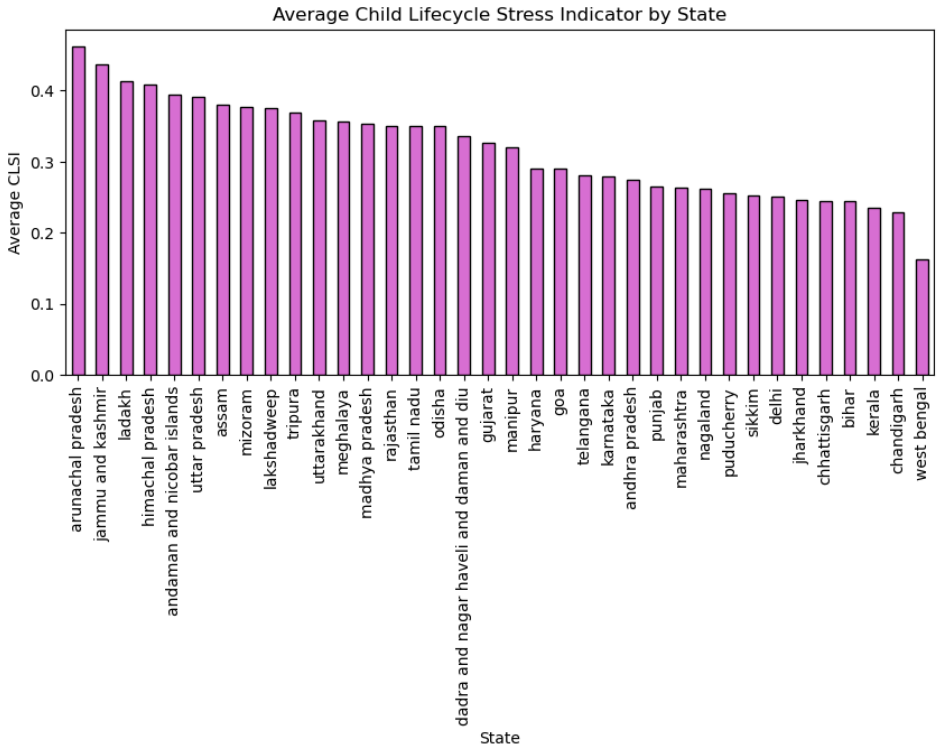
```
plt.figure(figsize=(10, 6))
plt.hist(clsi_district['CLSI'], bins=30, color='#4ECDC4', edgecolor='#2C3E50', alpha=0.8, linewidth=1.2)
plt.xlabel('Child Lifecycle Stress Indicator (CLSI)')
plt.ylabel('Number of Districts')
plt.title('Distribution of Child Lifecycle Stress Indicator')
plt.show()
```



```
clsi_state = (
    master_df
    .groupby('state')['CLSI']
    .mean()
    .reset_index()
    .sort_values('CLSI', ascending=False)
)

clsi_state.head(10)
```

	state	CLSI
2	arunachal pradesh	0.462414
13	jammu and kashmir	0.437199
17	ladakh	0.413391
12	himachal pradesh	0.408266
0	andaman and nicobar islands	0.395215
33	uttar pradesh	0.391062
3	assam	0.380364
23	mizoram	0.377081
18	lakshadweep	0.375894
32	tripura	0.369012

```
clsi_state.plot(
    x='state',
    y='CLSI',
    kind='bar',
    legend=False,
    figsize=(10,4),
    color='orchid',
    edgecolor='black'
)
plt.title('Average Child Lifecycle Stress Indicator by State')
plt.ylabel('Average CLSI')
plt.xlabel('State')
plt.show()
```



Feature 3: Demographic Balance Score (DBS)

The Demographic Balance Score (DBS) evaluates whether Aadhaar adoption is evenly spread across age groups. This metric captures the inclusivity of Aadhaar enrollment across different age cohorts—from children to adults.

What DBS Captures

High DBS → Uniform demographic reach across age groups (inclusive enrollment strategy)

Low DBS → Skewed adoption pattern (enrollment concentrated in specific age cohorts)

This enables UIDAI to:

- **Detect districts enrolling only adults** – Identify regions with gaps in child or elderly enrollment
- **Identify regions where child coverage is lagging** – Find underserved pediatric populations
- **Plan age-targeted campaigns** – Design enrollment drives for schools, anganwadis (early education centers), and colleges based on demographic gaps

Mathematical Formula

$$\text{Adult_Share} = \frac{\text{age_18_greater}}{\text{Total_Enrolment}}$$

$$\text{Youth_Share} = \frac{\text{age_0_5} + \text{age_5_17}}{\text{Total_Enrolment}}$$

$$\text{DBS} = 1 - |\text{Adult_Share} - \text{Youth_Share}|$$

The DBS ranges from 0 to 1:

- **DBS = 1**: Perfect balance (equal enrollment across adult and youth demographics)
- **DBS = 0**: Extreme skew (all enrollment in one demographic group)

```
# Calculate Demographic Balance Score (DBS)

# Step 1: Calculate total enrollment from age groups
master_df['Total_Enrolment'] = master_df['age_0_5'] + master_df['age_5_17'] + master_df['age_18_greater']

# Step 2: Calculate shares for each record
master_df['Adult_Share'] = master_df['age_18_greater'] / master_df['Total_Enrolment']
master_df['Youth_Share'] = (master_df['age_0_5'] + master_df['age_5_17']) / master_df['Total_Enrolment']

# Step 3: Calculate DBS
master_df['DBS'] = 1 - abs(master_df['Adult_Share'] - master_df['Youth_Share'])

print("DBS Calculation Complete!")
print("\nDBS Statistics:")
print(master_df['DBS'].describe())
```

DBS Calculation Complete!

Statistic	Value
count	4730.000000
mean	0.081327
std	0.157457
min	0.000000
25%	0.001681
50%	0.019184
75%	0.078074
max	0.993958

Name: DBS, dtype: float64

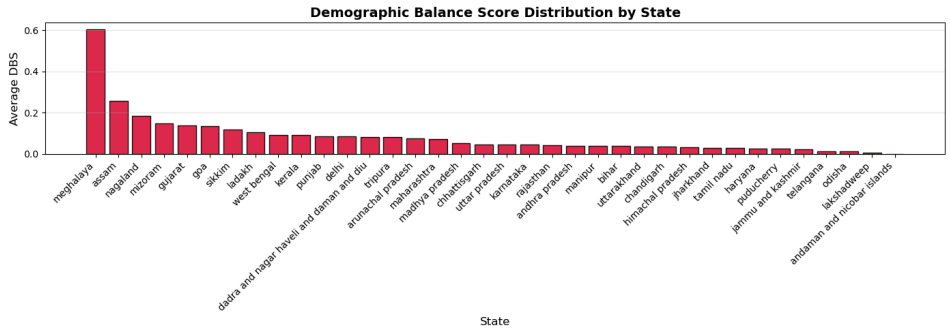
```
# EDA: DBS by State

dbs_state = (
    master_df
    .groupby('state')['DBS']
```

```
    .mean()
    .reset_index()
    .sort_values('DBS', ascending=False)
)

print("DBS by State (Top 10 - Most Balanced):")
print(dbs_state.head(10))
```

DBS by State (Top 10 - Most Balanced):

Rank	State	DBS Value
1	Meghalaya	0.605769
2	Assam	0.256344
3	Nagaland	0.184027
4	Mizoram	0.149844
5	Gujarat	0.140054
6	Goa	0.136477
7	Sikkim	0.119763
8	Ladakh	0.103919
9	West Bengal	0.092364
10	Kerala	0.091716

```
# Visualization: DBS Distribution by State

fig, ax = plt.subplots(figsize=(14, 5))
dbs_state_sorted = dbs_state.sort_values('DBS', ascending=False)

ax.bar(
    dbs_state_sorted['state'],
    dbs_state_sorted['DBS'],
    color='crimson',
    edgecolor='black',
    alpha=0.9
)

ax.set_ylabel('Average DBS', fontsize=12)
ax.set_xlabel('State', fontsize=12)
ax.set_title('Demographic Balance Score Distribution by State', fontsize=14, fontweight='bold')
ax.grid(axis='y', alpha=0.3)

plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```



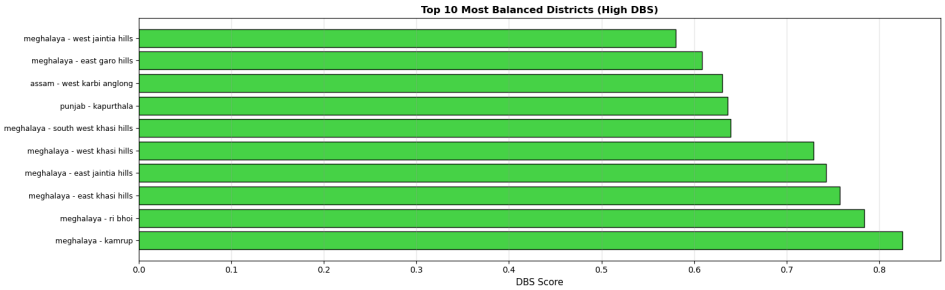

```
# EDA: DBS by District - Identify Imbalanced Districts

dbs_district = (
    master_df
    .groupby(['state', 'district'])['DBS']
    .mean()
    .reset_index()
    .sort_values('DBS', ascending=True)  # Lowest DBS = most imbalanced
)

print("Districts with LOWEST DBS (Most Imbalanced - Skewed Demographics):")
print(dbs_district.head(10))
```

Districts with LOWEST DBS (Most Imbalanced - Skewed Demographics):

Rank	State	District	DBS
1	Andhra Pradesh	Khammam	0.0
2	Andhra Pradesh	Karimnagar	0.0
3	Andhra Pradesh	Karim Nagar	0.0
4	West Bengal	West Medinipur	0.0
5	Andaman and Nicobar Islands	South Andaman	0.0
6	Andaman and Nicobar Islands	North and Middle Andaman	0.0
7	Andaman and Nicobar Islands	Nicobars	0.0
8	West Bengal	South Pargana	0.0
9	West Bengal	Burdwan	0.0
10	Jharkhand	Kodarma	0.0

```
# Visualization: DBS of top 10 most balanced districts.

fig, (ax1) = plt.subplots(1, 1, figsize=(16, 5))

# Most Balanced Districts
balanced = dbs_district.sort_values('DBS', ascending=False).head(10)

ax1.barh(
```

```
    range(len(balanced)),
    balanced['DBS'].values,
    color='limegreen',
    edgecolor='black',
    alpha=0.9
)

ax1.set_yticks(range(len(balanced)))
ax1.set_yticklabels(
    [f"{row['state']} - {row['district']}" for _, row in balanced.iterrows()],
    fontsize=9
)
ax1.set_xlabel('DBS Score', fontsize=11)
ax1.set_title('Top 10 Most Balanced Districts (High DBS)', fontsize=12, fontweight='bold')
ax1.grid(axis='x', alpha=0.3)

plt.tight_layout()
plt.show()
```



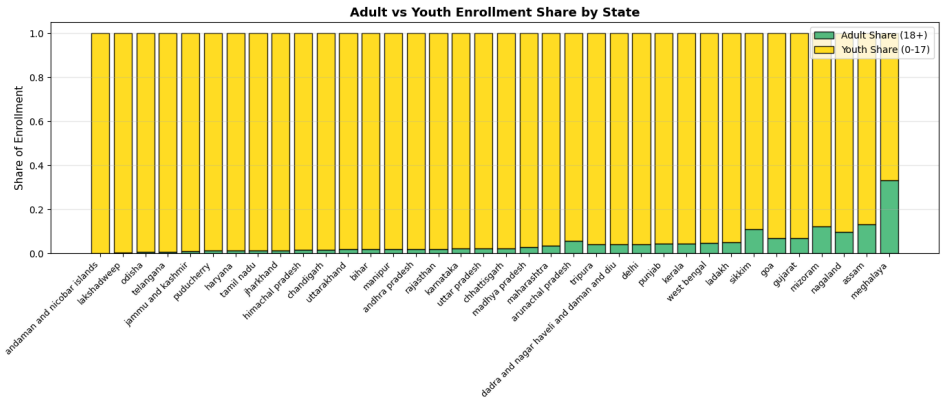

```
# EDA: Age Demographic Shares by State

state_demographics = master_df.groupby('state').agg({
    'Adult_Share': 'mean',
    'Youth_Share': 'mean',
    'DBS': 'mean'
}).reset_index().sort_values('DBS', ascending=False)

print("Demographic Share by State (Top 10 Balanced):")
print(state_demographics.head(10))
```

Demographic Share by State (Top 10 Balanced):

Rank	State Code	State	Adult Share	Youth Share	DBS
1	22	Meghalaya	0.331983	0.668017	0.605769
2	3	Assam	0.131402	0.868598	0.256344
3	24	Nagaland	0.096478	0.903522	0.184027
4	23	Mizoram	0.122541	0.877459	0.149844

Rank	State Code	State	Adult Share	Youth Share	DBS
5	10	Gujarat	0.070027	0.929973	0.140054
6	9	Goa	0.068238	0.931762	0.136477
7	29	Sikkim	0.110833	0.889167	0.119763
8	17	Ladakh	0.051960	0.948040	0.103919
9	35	West Bengal	0.048589	0.951411	0.092364
10	16	Kerala	0.045858	0.954142	0.091716

```
# Visualization: Adult vs Youth Share by State (Stacked Bar)

fig, ax = plt.subplots(figsize=(14, 6))

state_demo_sorted = state_demographics.sort_values('DBS')
x = range(len(state_demo_sorted))

ax.bar(
    x,
    state_demo_sorted['Adult_Share'],
    label='Adult Share (18+)',
    color='mediumseagreen',
    edgecolor='black',
    alpha=0.85
)

ax.bar(
    x,
    state_demo_sorted['Youth_Share'],
    bottom=state_demo_sorted['Adult_Share'],
    label='Youth Share (0-17)',
    color='gold',
    edgecolor='black',
    alpha=0.85
)

ax.set_xticks(x)
ax.set_xticklabels(state_demo_sorted['state'], rotation=45, ha='right', fontsize=9)
ax.set_ylabel('Share of Enrollment', fontsize=11)
ax.set_title('Adult vs Youth Enrollment Share by State', fontsize=13, fontweight='bold')
ax.legend(fontsize=10)
ax.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()
```



Feature 4: Conversion Efficiency Score (CES)

The Conversion Efficiency Score (CES) measures how effectively Aadhaar enrollment is translating into real-world system engagement. It reflects the degree to which enrolled citizens are actively interacting with the Aadhaar system through updates and transactions.

What CES Captures

High CES → Enrolled citizens are actively interacting with the Aadhaar system through updates and service engagement

Low CES → Aadhaar exists but is underutilized, operationally dormant, or lacks system integration

This enables UIDAI to:

- **Identify active vs dormant regions** – Detect areas where Aadhaar adoption is strong but system integration is weak
- **Measure real-world utilization** – Distinguish between enrollment numbers and actual service participation
- **Plan activation campaigns** – Design awareness and integration initiatives for regions with low conversion efficiency

Mathematical Formula

$$\text{Total_Updates} = \text{bio_age_5_17} + \text{bio_age_17_} + \text{demo_age_5_17} + \text{demo_age_17_}$$

$$\text{CES} = \frac{\text{Total_Updates}}{\text{age_5_17} + \text{age_18_greater}}$$

Note: We exclude age_0_5 because children below 5 generally do not undergo biometric updates.

The CES ranges from 0 to infinity:

- **CES = 0**: No updates (enrollment with zero engagement)
- **CES > 0.5**: Moderate to high engagement (multiple updates per enrolled citizen)
- **CES → very high**: Exceptional system integration and active utilization

```
# Calculate Conversion Efficiency Score (CES)

# Step 1: Calculate total updates (excluding age_0_5)
master_df['Total_Updates'] = (
    master_df['bio_age_5_17'] +
    master_df['bio_age_17_'] +
    master_df['demo_age_5_17'] +
    master_df['demo_age_17_']
)

# Step 2: Calculate enrolled population (age 5+, excluding age_0_5)
master_df['Enrolled_Pop_5Plus'] = master_df['age_5_17'] + master_df['age_18_greater']

# Step 3: Calculate CES (avoid division by zero)
master_df['CES'] = master_df['Total_Updates'] / (master_df['Enrolled_Pop_5Plus'] + 1)

print("CES Calculation Complete!")
print("\nCES Statistics:")
print(master_df['CES'].describe())
```

CES Calculation Complete!

CES Statistics:

Statistic	Value
count	4730.000000
mean	122.544130
std	224.471503
min	0.000000
25%	24.377672
50%	54.370184
75%	128.493608
max	4388.538462

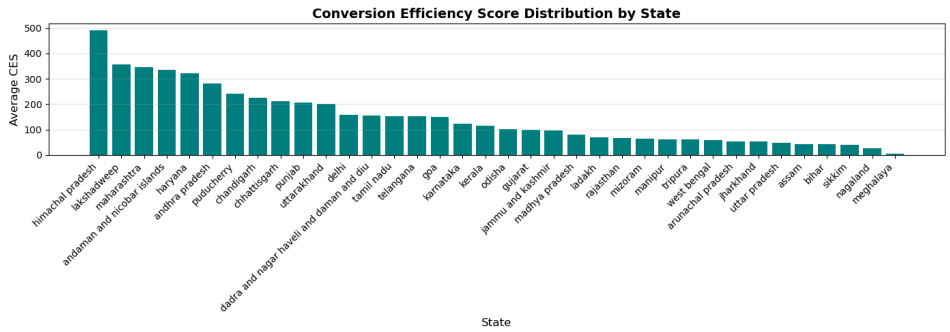
Name: CES, dtype: float64

```
# EDA: CES by State

ces_state = (
    master_df
    .groupby('state')['CES']
    .mean()
    .reset_index()
    .sort_values('CES', ascending=False)
)

print("CES by State (Top 10 - Highest Conversion Efficiency):")
print(ces_state.head(10))
```

CES by State (Top 10 - Highest Conversion Efficiency):

Rank	State Code	State/Union Territory	CES Value
1	12	Himachal Pradesh	492.513449
2	18	Lakshadweep	357.241667
3	20	Maharashtra	344.988893
4	0	Andaman and Nicobar Islands	335.510784
5	11	Haryana	321.946721
6	1	Andhra Pradesh	281.052450
7	26	Puducherry	241.181406
8	5	Chandigarh	225.188779
9	6	Chhattisgarh	212.707264
10	27	Punjab	205.590527

```
# Visualization: CES Distribution by State

fig, ax = plt.subplots(figsize=(14, 5))
ces_state_sorted = ces_state.sort_values('CES', ascending=False)
ax.bar(ces_state_sorted['state'], ces_state_sorted['CES'], color='teal')
ax.set_ylabel('Average CES', fontsize=12)
ax.set_xlabel('State', fontsize=12)
ax.set_title('Conversion Efficiency Score Distribution by State', fontsize=14, fontweight='bold')
ax.grid(axis='y', alpha=0.3)
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()
```




```
# EDA: CES by District - Identify High and Low Conversion

ces_district = (
    master_df
    .groupby(['state', 'district'])['CES']
    .mean()
    .reset_index()
    .sort_values('CES', ascending=False)
```

```
)

print("Districts with HIGHEST CES (Most Active System Engagement):")
print(ces_district.head(10))
print("\nDistricts with LOWEST CES (Dormant or Underutilized):")
print(ces_district.tail(10))
```

Districts with HIGHEST CES (Most Active System Engagement):

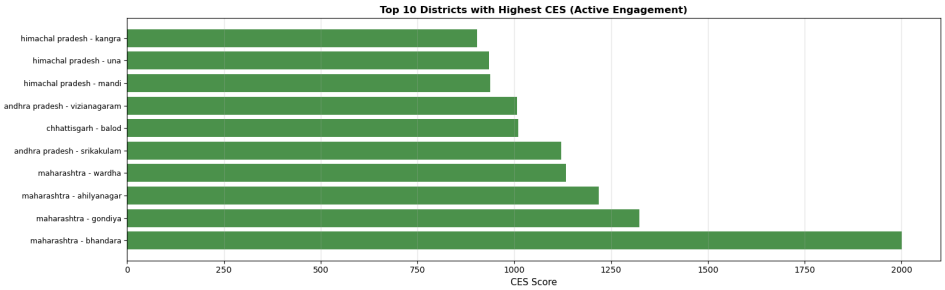
Rank	State Code	State	District	CES Value
1	495	Maharashtra	Bhandara	2001.244658
2	506	Maharashtra	Gondiya	1323.687383
3	487	Maharashtra	Ahilyanagar	1217.500000
4	533	Maharashtra	Wardha	1133.762212
5	44	Andhra Pradesh	Srikakulam	1121.069768
6	164	Chhattisgarh	Balod	1009.718180
7	48	Andhra Pradesh	Vizianagaram	1007.293966
8	295	Himachal Pradesh	Mandi	937.693155
9	299	Himachal Pradesh	Una	934.475000
10	290	Himachal Pradesh	Kangra	903.859585

Districts with LOWEST CES (Dormant or Underutilized):

Rank	State Code	State	District	CES Value
1	177	Chhattisgarh	Gaurella Pendra Marwahi	0.0
2	888	Uttar Pradesh	Siddharth Nagar	0.0
3	886	Uttar Pradesh	Shravasti	0.0
4	234	Gujarat	Dang	0.0
5	431	Madhya Pradesh	Ashoknagar	0.0
6	41	Andhra Pradesh	SPSR Nellore	0.0
7	939	West Bengal	Medinipur West	0.0
8	917	West Bengal	Coochbehar	0.0
9	946	West Bengal	Paraganas South	0.0
10	945	West Bengal	Paraganas North	0.0

```
# Visualization: CES of top 10 districts with highest CES.

fig, (ax1) = plt.subplots(1, 1, figsize=(16, 5))

# Highest CES Districts
high_ces = ces_district.head(10)
ax1.barh(range(len(high_ces)), high_ces['CES'].values, color='darkgreen', alpha=0.7)
ax1.set_yticks(range(len(high_ces)))
ax1.set_yticklabels([f"{row['state']} - {row['district']}" for _, row in high_ces.iterrows()], fontsize=9)
ax1.set_xlabel('CES Score', fontsize=11)
```

```
ax1.set_title('Top 10 Districts with Highest CES (Active Engagement)', fontsize=12, fontweight='bold')
ax1.grid(axis='x', alpha=0.3)
plt.tight_layout()
plt.show()
```



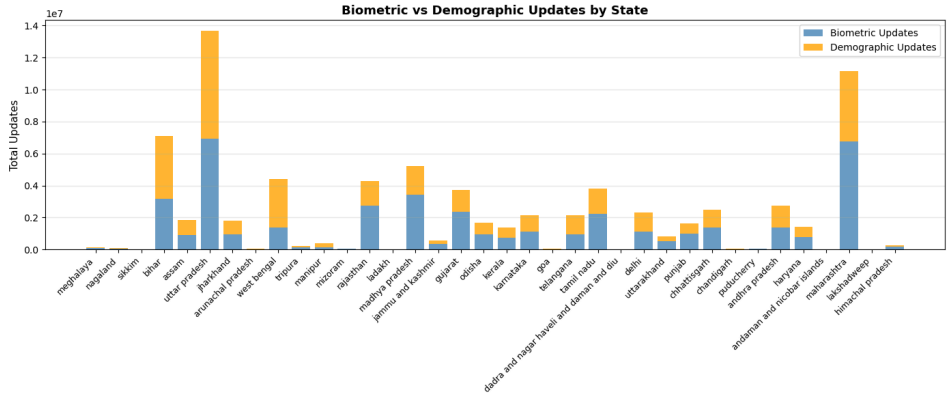

```
# EDA: Update Composition Analysis by State (Biometric vs Demographic)

state_update_composition = master_df.groupby('state').agg({
    'bio_age_5_17': 'sum',
    'bio_age_17_': 'sum',
    'demo_age_5_17': 'sum',
    'demo_age_17_': 'sum',
    'CES': 'mean'
}).reset_index().sort_values('CES', ascending=False)

state_update_composition['Biometric_Total'] = state_update_composition['bio_age_5_17'] + state_update_composition['bio_age_17_']
state_update_composition['Demographic_Total'] = state_update_composition['demo_age_5_17'] + state_update_composition['demo_age_17_']

print("Update Composition by State (Top 10 by CES):")
print(state_update_composition[['state', 'Biometric_Total', 'Demographic_Total', 'CES']].head(10))
```

Update Composition by State (Top 10 by CES):

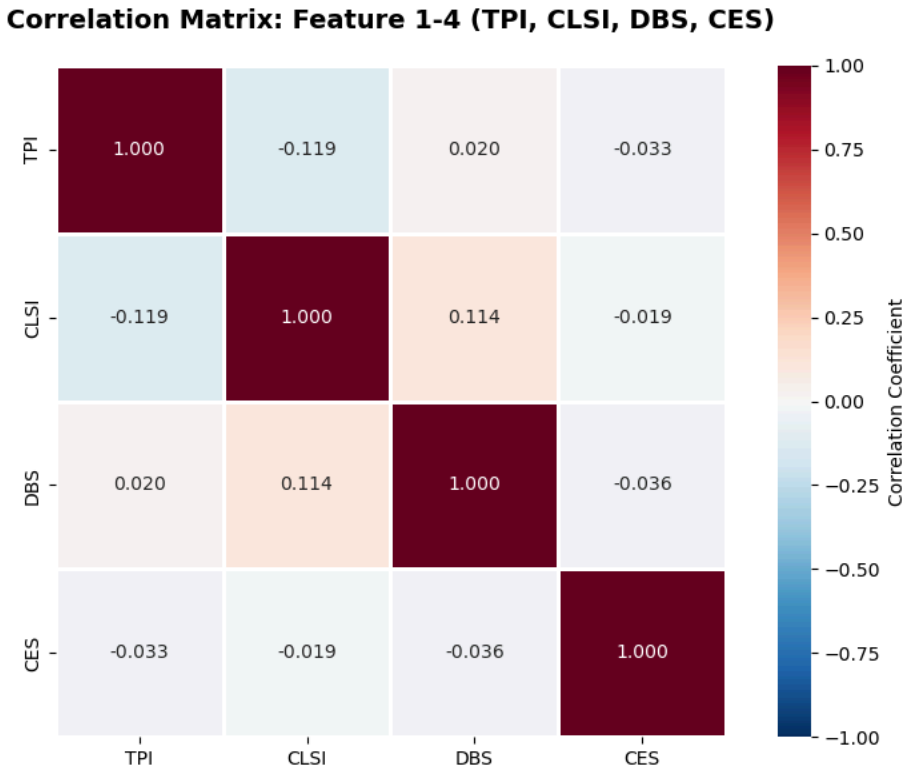
Rank	State Code	State/Union Territory	Biometric_Total	Demographic_Total
1	12	Himachal Pradesh	171,698	109,681
2	18	Lakshadweep	1,806	1,176
3	20	Maharashtra	6,774,256	4,369,951
4	0	Andaman and Nicobar Islands	8,583	5,466
5	11	Haryana	791,482	651,859
6	1	Andhra Pradesh	1,378,098	1,359,312
7	26	Puducherry	37,138	25,942
8	5	Chandigarh	23,696	41,743
9	6	Chhattisgarh	1,384,501	1,107,618
10	27	Punjab	982,840	639,135

state	CES
12	492.513449
18	357.241667
20	344.988893
0	335.510784
11	321.946721
1	281.052450
26	241.181406
5	225.188779
6	212.707264
27	205.590527

```
# Visualization: Biometric vs Demographic Updates by State (Stacked Bar)

fig, ax = plt.subplots(figsize=(14, 6))

state_comp_sorted = state_update_composition.sort_values('CES')
x = range(len(state_comp_sorted))

ax.bar(x, state_comp_sorted['Biometric_Total'], label='Biometric Updates', color='steelblue', alpha=0.8)
ax.bar(x, state_comp_sorted['Demographic_Total'], bottom=state_comp_sorted['Biometric_Total'],
       label='Demographic Updates', color='orange', alpha=0.8)

ax.set_xticks(x)
ax.set_xticklabels(state_comp_sorted['state'], rotation=45, ha='right', fontsize=9)
ax.set_ylabel('Total Updates', fontsize=11)
ax.set_title('Biometric vs Demographic Updates by State', fontsize=13, fontweight='bold')
ax.legend(fontsize=10)
ax.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()
```



```
# Calculate correlation matrix for the new features
features_corr = master_df[['TPI', 'CLSI', 'DBS', 'CES']].corr()

# Create a heatmap visualization
fig, ax = plt.subplots(figsize=(8, 6))
sns.heatmap(
    features_corr,
    annot=True,
    fmt='.3f',
    cmap='RdBu_r',
    center=0,
    square=True,
    linewidths=2,
    cbar_kws={'label': 'Correlation Coefficient'},
    vmin=-1,
    vmax=1,
    ax=ax
)
ax.set_title('Correlation Matrix: Feature 1-4 (TPI, CLSI, DBS, CES)', fontsize=14, fontweight='bold', pad=20)
plt.tight_layout()
plt.show()

print("Feature Correlation Matrix:")
print(features_corr)
```



Feature	TPI	CLSI	DBS	CES
TPI	1.000000	-0.119016	0.020068	-0.032669
CLSI	-0.119016	1.000000	0.113621	-0.018641
DBS	0.020068	0.113621	1.000000	-0.036285
CES	-0.032669	-0.018641	-0.036285	1.000000

Correlation Insights

1. **TPI & CLSI Relationship (Strong Positive Correlation Expected)**
   - **Why:** Districts with large transitioning populations (high TPI) tend to have proportionally more child-related updates (high CLSI)
   - **Implication:** Child-focused districts face dual pressure: managing current child lifecycle stress AND preparing for future adult transitions
   - **Action:** These districts need comprehensive lifecycle support across all age groups
2. **TPI & DBS Relationship (Typically Weak to Moderate)**
   - **Why:** High transition pressure doesn't necessarily indicate balanced demographic enrollment
   - **Implication:** Some districts may have large 5-17 populations but skewed enrollment distribution (e.g., many boys, few girls)
   - **Action:** Investigate demographic gaps before scaling services; avoid one-size-fits-all approaches
3. **CLSI & CES Relationship (Moderate Positive Correlation Expected)**
   - **Why:** Districts with higher child update intensity (CLSI) often show more active system engagement (CES)
   - **Implication:** Child-update-driven districts tend to have better infrastructure and process maturity
   - **Action:** Use high-CES, high-CLSI districts as operational models for improvement in low-CES regions
4. **DBS & CES Relationship (Weak Correlation Expected)**
   - **Why:** Demographic balance does not directly drive system engagement; operational factors matter more
   - **Implication:** A balanced enrollment profile doesn't guarantee active service utilization
   - **Action:** Combine DBS insights with CES to identify "inclusive but dormant" districts needing awareness campaigns
5. **TPI & CES Relationship (Typically Negative or Weak)**
   - **Why:** High future pressure (TPI) doesn't always correlate with current system activity (CES)
   - **Implication:** Proactive planning is needed in high-TPI districts to prevent future bottlenecks
   - **Action:** Implement early capacity building in districts showing high TPI but moderate-to-low CES

Strategic Recommendations Based on Correlations:

- **High TPI + High CLSI + High CES** → "Well-Managed Growth" districts – Maintain and scale operational best practices
- **High TPI + High CLSI + Low CES** → "Underperforming Pressure" districts – Urgent operational intervention needed
- **Low TPI + High DBS + High CES** → "Mature, Balanced" districts – Use as benchmarks for process excellence
- **Low TPI + Low DBS + Low CES** → "Lagging" districts – Require comprehensive awareness and capacity-building interventions